ID2203 Project Report: Battle-testing OmniPaxos

Minyi Zhu
minyizhu@kth.se

Zhouya Shen
zhouya@kth.se

Songyan Lai
songyanl@kth.se

March 16, 2026

Abstract

This report describes the design and implementation of a robust black-box testing framework for OmniPaxos, a distributed consensus algorithm. As part of the ID2203 Distributed Systems Advanced Course Project, we battle-tested an OmniPaxos-supported key-value store using Maelstrom, a Jepsen-style testing framework. We added to the existing codebase by developing a testable HTTP shim, a protocol adapter, and nemesis scripts to introduce aggressive network partitions and node failures. To guarantee strict linearizability, we developed a "read via log" approach, which removed stale reads during leader isolation. Our empirical assessment in the presence of extreme faults shows that the OmniPaxos implementation correctly maintains linearizability. Moreover, we assessed correct state recovery after process restarts using a RocksDB-based persistent storage backend. In summary, our findings empirically validate the correctness and safety of the OmniPaxos implementation in the presence of aggressive adversarial attacks.

Our code is open-sourced on https://github.com/Zeyashen/omnipaxos-kv-final.

1 Introduction

This project is a part of the ID2203 VT26 Distributed Systems Advanced Course Project and is a response to the assignment "Battle-testing OmniPaxos with Jepsen."

Distributed consensus protocols such as Paxos and Raft form the foundation of contemporary distributed systems and provide high consistency guarantees even when things go awry. However, there is often a mismatch between the theoretical correctness of the protocol and the way it actually behaves in the code. Tiny bugs that seep into the code usually manifest only in very specific and aggressive fault scenarios.To bridge this gap, rigorous black-box testing and fault injection frameworks, such as Jepsen [1] and Maelstrom [2], have become the industry standard for validating distributed databases.

In this work, we developed a distributed in-memory key-value store with linearizable operations to empirically verify the resilience of the OmniPaxos consensus algorithm. Our system is written in Rust and layers on top of the OmniPaxos consensus algorithm to maintain consistency of the replicated state across all nodes. We forked the original GitHub[1] repository and introduced a testable HTTP shim, Maelstrom adapter, and nemesis-driven fault injection to explore partitions and crash recovery. The main hypothesis of our work is that the OmniPaxos implementation is strictly linearizable even in the presence of extreme network partitions and node crashes.

The balance of the rest of this report is organized as follows. In Section 2, we describe the system at a high level and the models that underlie our work. Section 3 describes the architecture and the rationale for our design decisions, including how we implement linearizable reads. Section 4 describes our implementation in more detail, as it relates to the project requirements, including our approach to reconnecting the shim and to storage. Section 5 and

[1] https://github.com/haraldng/omnipaxos-kv

Section 6 describe the experimental setup and then the analysis of the results of our verification. Finally, Section 7 discusses the challenges we faced and future directions, with a summary in Section 8.

2 System Overview and Model

2.1 Overview

Our system is a distributed in-memory key-value store implemented on top of OmniPaxos. Client operations are translated to commands, which are appended to a replicated log and then executed in the decided order on each replica's local state machine. In this way, each replica executes the same order of commands and reaches the same state.

The configuration consists of three OmniPaxos servers and an HTTP shim for handling requests and a Maelstrom adapter for workload generation and result validation. The shim is responsible for handling incoming requests(`put/get/cas`) and returning the results back to the client. This design provides a clean separation between the benchmarking interface and the replication engine.

2.2 System Model

We evaluate the system in an asynchronous environment where delays can be arbitrary and failures can happen at any time. We use a crash-recovery model and simulate network partitions at the Docker network level. In this environment, maintaining linearizability requires that all completed operations seem to occur instantaneously and in a single global order that also respects real-time order.

To mitigate stale reads during failures, we use read-via-log, where all reads go through the replicated log. This design trade-off introduces some latency in reads but provides stronger and easier-to-reason-about consistency guarantees during network partitions and leader transitions.

2.2.1 Assumptions

We use reliable point-to-point transport for established connections (TCP), non-Byzantine nodes, and deterministic state-machine execution on replicas. The experiments describe a fixed 3-node cluster configuration and are not targeting WAN-scale deployment or Byzantine attackers in this research project.

2.3 Verification Approach

Instead of formally verifying protocol correctness, we use Jepsen-style black-box testing with Maelstrom. Maelstrom provides a concurrent operation generator, nemesis modules for fault injection, and a linearizability checker. This allows us to test the entire implementation stack, including protocol adaptation, timeouts, and recovery logic.

3 System Architecture and Design

3.1 High-level Architecture

As illustrated in Figure 1, the pipeline securely bridges the Maelstrom testing framework and the OmniPaxos KV cluster. The Protocol Bridge layer intercepts standard JSON-formatted operations, translates them into HTTP REST requests, and manages timeout semantics by mapping them to Maelstrom's indeterminate state. Nemesis scripts act as an external adversarial agent, periodically injecting crash-recovery and network partition faults into the cluster.

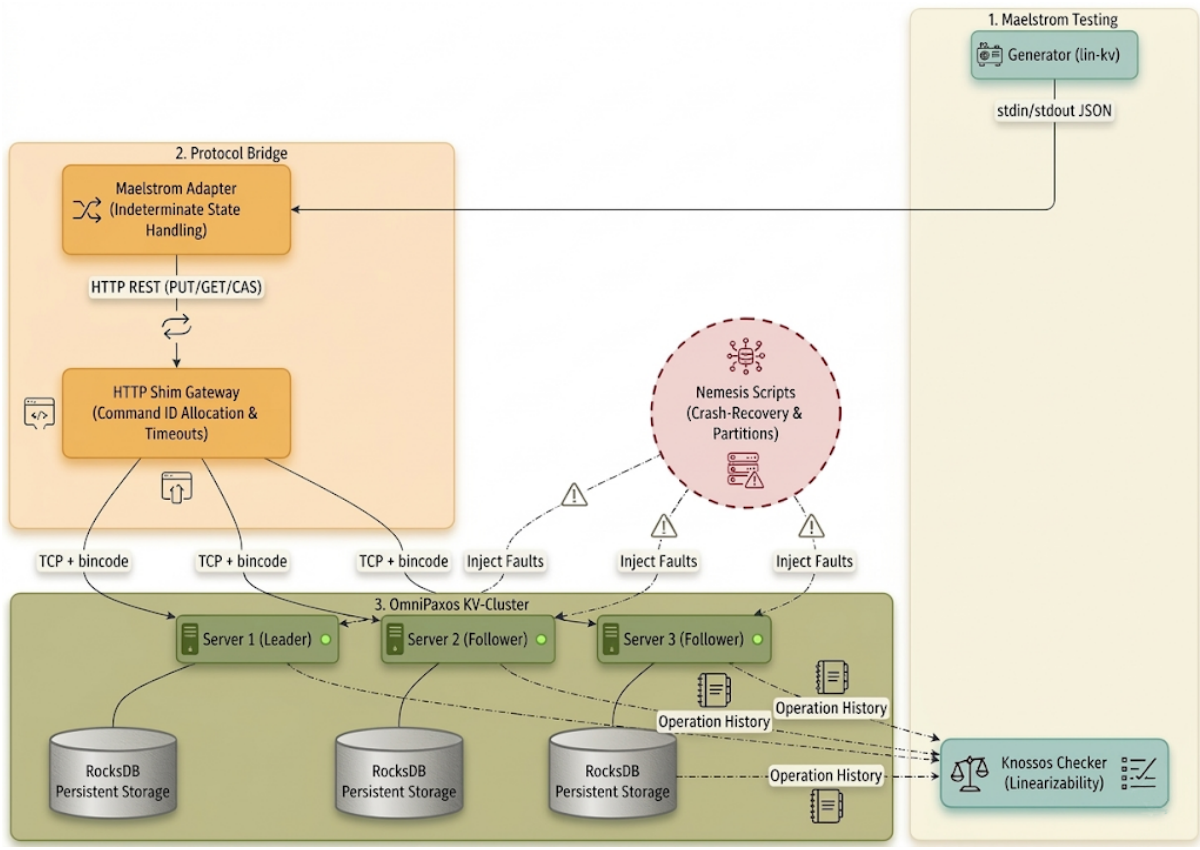

Figure 1: End-to-End Verification Pipeline and System Architecture.

3.2 Design Rationale

3.2.1 HTTP Shim Design

We picked HTTP for the shim layer primarily for its observability and interoperability. It allows us to poke at the KV store using common tools (such as curl) during manual debugging, and it provides a clean architectural separation between the external benchmark protocols and our internal Rust state machine.

3.2.2 Dedicated Maelstrom Adapter

Maelstrom communicates exclusively over standard I/O (JSON), while our shim speaks HTTP REST. A separate adapter ensures that both interfaces stay clean and focused on a single responsibility. Crucially, it bundles the difficult part of fault semantics and timeouts into a single modular unit.

3.2.3 Indeterminate Outcome Classification

In a distributed system with partition faults, a network timeout is necessarily ambiguous: it could indicate that the message never reached the leader, or that the leader did in fact record the operation but that the acknowledgement was lost. If we had simply logged these as "failures," the Knossos checker might have incorrectly identified a linearizability violation based on that information when it appeared in the system later [3]. By systematically translating these into Maelstrom's indeterminate state (error code 14), we ensure that the mathematical integrity of the checker's history validation is maintained.

3.2.4 Read-via-Log for Linearizable Reads

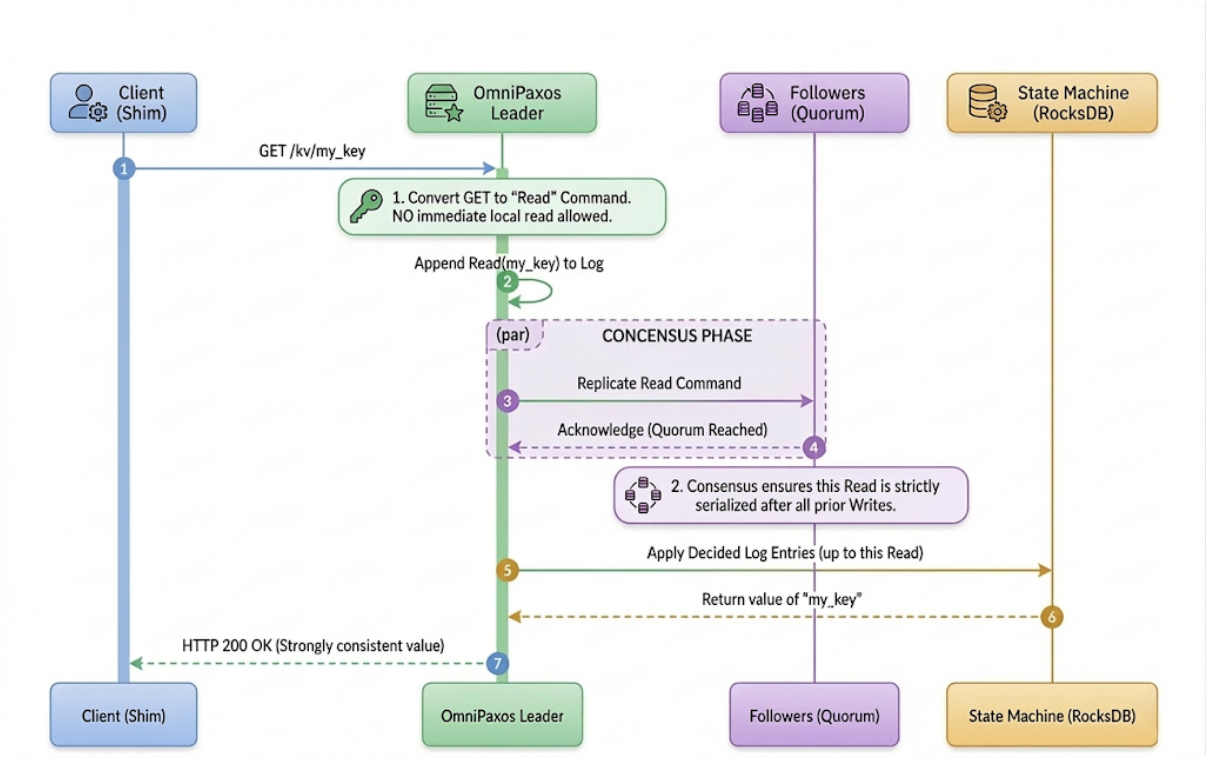

Figure 2: The Read-via-Log mechanism.

In a consensus system, a local read can very easily perform stale reads. Consider a leader who is in a minority partition and is not aware of being overthrown—the local read can return stale values, violating linearizability. To correct this, we modified the read path to append a Read operation directly to the OmniPaxos log. The "read via log" approach is slower since reads also require a quorum like writes, but it ensures that all reads are tied to writes and that there are no stale reads even in the face of aggressively splitting networks.

3.3 Message and Command Model

In order to connect the Maelstrom adapter's synchronous HTTP calls to the OmniPaxos's asynchronous consensus loop, we established a common communication protocol in `src/common.rs`. The essential data components are `KVCommand`, which is an enum representing the deterministic state machine steps (`Put, Get, Delete, Cas`); `ClientMessage`, which is composed of a `KVCommand` and a monotonically increasing `CommandId`; and `ServerMessage`, which is sent by the server's state machine loop back to the Shim with the result correlated to the `CommandId`. The `CommandId` is generated on the fly by the Shim when it receives an external request.

The addition of a command to the OmniPaxos log is inherently an asynchronous process, where replication, quorum decisions, and state machine execution must all occur before the result is available. The HTTP Shim must park the incoming request. The `CommandId` becomes the essential correlation key. When the Shim sends the `ClientMessage` to the server, it registers a Tokio `oneshot` channel keyed by the `CommandId`. The server returns the `ServerMessage` on the registered `oneshot` channel after applying the chosen log entry to its state machine loop, allowing the Shim to unblock and send the HTTP response to the client.

3.4 End-to-End Request Flow

3.4.1 Write and CAS Operations

When the Maelstrom generator sends a `write(k,v)` or `cas(k, current, new)` request, the adapter intercepts and translates it into an HTTP `PUT` or `POST /kv/cas` request, respectively. Upon receiving this, the Shim allocates a unique `CommandId`, wraps the operation in a `ClientMessage`, and sending it to the OmniPaxos cluster, while waiting asynchronously on a local oneshot channel for the result.

On the server side, the coordinating app adds a command to the replicated log, waits for a quorum to agree on the log entry, applies the command to the deterministic state machine, which executes the operation on the RocksDB state machine, sending a `ServerMessage` with the result of the execution back to the Shim through a correlation ID, with the Shim responding with the appropriate HTTP status, which in turn causes the adapter to send a `write_ok`, `cas_ok`, or error back to the Maelstrom checker.

3.4.2 Read Operations and Linearizability

As elaborated in our design rationale, read operations do not bypass the consensus layer. A Maelstrom `read(k)` request is translated into a `GET` HTTP call. The Shim wraps this in a `Read` command and submits it to the OmniPaxos log. The system ensures strict serialization of reads in the global consensus order by making them pass through the same append and decide process as writes. This is achieved by Read-via-Log. This ensures no stale reads when partitions happen.

4 Implementation Details

4.1 Requirement Mapping

Table 1: Requirement-to-implementation mapping

Requirement	**Main files**	**Implemented behavior**
Testable shim	`src/shim/main.rs`	HTTP API for put/get/cas, command-id allocation, timeout/disconnect classification
Client + generator bridge	`src/maelstrom_node/main.rs`	Maelstrom protocol adapter, error mapping, cas failure classification
Nemesis fault injection	`build_scripts/nemesis.sh`, `run-nemesis-test.sh`	Partition/crash/mixed loops with scripted orchestration
Linearizability verification	Maelstrom `lin-kv` + checker	History validation under no-fault and fault scenarios
Bonus recovery	`src/server/server.rs`, `verify-recovery.sh`, compose file	Persistent storage + crash-restart durability checks

4.2 Step 1: Testable Shim

The core structure of the testable shim relies on maintaining a pending map for asynchronous request handling. The timeout path is explicitly caught and translated to `indeterminate=true`, which preserves the uncertainty needed by the checker.

```
[language=Rust]
struct ShimClient {
    next_command_id: AtomicUsize,
    request_timeout: Duration,
    to_server: mpsc::Sender<ClientMessage>,
    pending: Arc<Mutex<HashMap<CommandId, oneshot::Sender<ServerMessage>>>>,
}

match timeout(self.request_timeout, rx).await {
    Ok(Ok(response)) => Ok(response),
    Ok(Err(_)) => Err(ShimError::Disconnected),
    Err(_) => Err(ShimError::Timeout),
}
```

4.3 Step 2: Maelstrom Adapter and Generator

The adapter handles four main operation types: `init` for node bootstrap reply, `read` for GET conversion, `write` for PUT conversion, and `cas` for POST conversion plus failure classification. If the underlying request returns an indeterminate error (e.g., due to a timeout), the adapter utilizes Maelstrom's error code 14 to properly classify the outcome.

```
[language=Rust]
if err.indeterminate.unwrap_or(false) {
    self.indeterminate_reply(req, "indeterminate read"); // code 14
}
```

4.4 Step 3: Nemesis Fault Injection Implementation

The Nemesis is the adversarial core of our testing pipeline, responsible for injecting systematic failures to challenge the cluster's consistency guarantees. We implemented two distinct fault classes using shell-scripted orchestration targeting the Docker daemon.

For Network Partitions (Split-Brain Scenarios), we manipulate the container network interfaces to simulate severe network degradation. We implemented three partition topologies: `isolate-leader`, which cleanly severs the current leader's network connections from followers; `majority-minority`, which slices the 3-node cluster into a 2-node active partition and a 1-node isolated partition; and `random-halves`, which randomly drops packets or disconnects nodes.

For Node Crashes and Restarts (Crash-Recovery), we simulate catastrophic process failures using the `docker kill` command. A major technical challenge we faced was handling the scenario where the OmniPaxos leader crashes, especially since our HTTP shim is typically pinned to a specific node (e.g., s1) to forward requests. When the Nemesis script kills s1, the underlying TCP connection between the shim and s1 is abruptly terminated. To handle this gracefully, the shim detects the EOF/disconnect from the TCP socket and immediately maps any pending asynchronous requests to an indeterminate state. Furthermore, our orchestration script incorporates a shim restart handling logic. When the killed node is brought back online via `docker start`, the shim also re-establishes the connection, ensuring that the benchmark can seamlessly resume sending traffic without manual intervention.

4.5 Step 4: Linearizability Verification

No custom checker was required. Correctness depended entirely on our accurate mapping of operation outcomes. Successful operations are reported as success, semantic failures (e.g., CAS mismatch) are reported as failures, and uncertain outcomes (timeouts) are mapped to indeterminate.

4.6 Step 5 (Bonus): Persistent Storage and Recovery

To fulfill the bonus requirement of recovery verification, we replaced the in-memory storage with a persistent key-value store. Specifically, we utilized the `omnipaxos_storage` crate enabled with the `persistent_storage` feature flag. This integration allows the OmniPaxos log and internal state to be flushed to a RocksDB instance on disk.

```
[language=Rust]
let storage_config = PersistentStorageConfig::with_path(storage_path);
let storage = PersistentStorage::<Command>::open(storage_config);
let omnipaxos = omnipaxos_config.build(storage).unwrap();
```

We configured Docker volumes in the `docker-compose-nemesis.yml` file to mount these storage paths directly to the host. Consequently, when a Docker container is forcefully killed and later restarted by the Nemesis script, the new instance can reconstruct its state machine and the replicated log from the persistent RocksDB volumes, ensuring that no committed entries are lost during the crash-recovery cycle.

5 Experimental Setup and Methodology

5.1 Environment

Our experimental setup relies on Docker and docker-compose to orchestrate the 3-node OmniPaxos cluster alongside the shim. The implementation is written in Rust, utilizing Tokio for asynchronous runtimes, while the testing harness is driven by shell scripts and Maelstrom's `lin-kv` workload. The `lin-kv` workload specifically generates a mix of read, write, and CAS operations to a set of independent keys, which are then analyzed against a strict-serializable consistency model.

5.2 Campaign Parameters and Scenarios

The campaign parameters were carefully chosen to stress the system. For the baseline runs, we utilized a time limit of 20 seconds, an operation rate of 50 requests per second, and a concurrency level of 6 workers. During the Nemesis runs, the time limit was extended to 30 seconds with fault injection occurring every 8 seconds and lasting for 6 seconds.

Our scenario design encompasses a baseline check with no faults, a partition-only test, a crash-only test, a mixed fault test (combining partitions and crashes), and a dedicated bonus recovery verification. The primary metric for evaluation is the strict linearizability verdict returned by the Knossos checker. Secondary metrics include the counts of `ok`, `fail`, and `info` outcomes, alongside the overall availability of the cluster under stress.

Table 2: Scenario-level verification outcomes

Scenario	isvalid	linearizable	ok-count	fail-count	info-count
Baseline	true	true	723	247	0
Partition-only	true	true	901	379	9
Crash-only	true	true	1028	396	0
Mixed	true	true	1087	387	0

Table 3: Availability comparison

Scenario	availability (%)
Baseline	74.53
Partition-only	69.89
Crash-only	72.19
Mixed	73.74

6 Results and Analysis

6.1 Overall Summary

6.2 Data Visualization

To better illustrate the system's performance under different fault conditions, we visualized the tabular data. Figure 3 highlights the ratio of successful operations (`ok-count`) to rejected operations (`fail-count`). Figure 4 demonstrates the slight drop in overall system availability when fault injections (like network partitions) force the cluster to prioritize safety over availability.

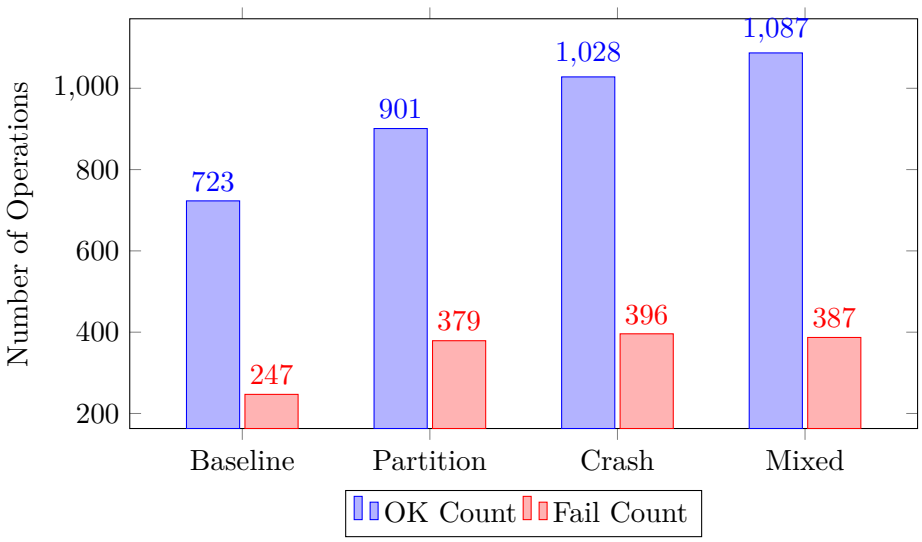

Figure 3: Operation Outcomes (OK vs. Fail) across Test Scenarios.

6.3 Result Interpretation

The most critical outcome is that the system remained `linearizable (true)` across all test scenarios, confirming the robustness of the OmniPaxos implementation under adversarial conditions.

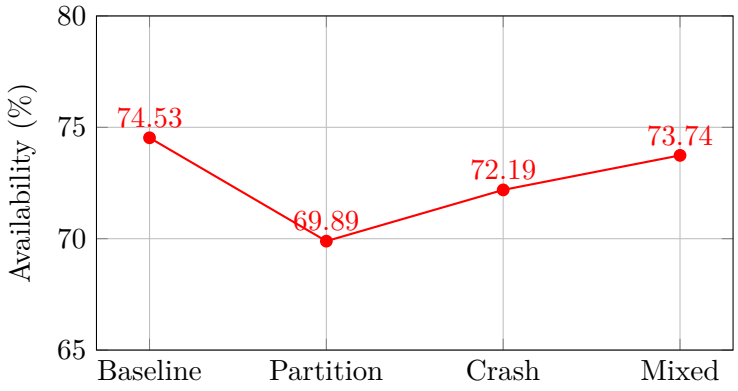

Figure 4: Availability Comparison across Test Scenarios.

As shown in Figure 3, we have logged a visible number of failed operations. It is worth mentioning that these are not indicative of bugs in the system. In a very concurrent test environment with six concurrent workers, multiple Maelstrom instances will concurrently attempt Compare-And-Swap (CAS) on the same key at the same time. Because of the strict serialization enforced by OmniPaxos, only the first operation that satisfies the expected precondition will succeed, and the rest will fail the precondition check on purpose. Thus, the failures actually verify that our concurrency mechanisms are working correctly to prevent malicious changes to the state.

Moreover, in the test case of Partition-only, the info-count is 9 operations. These are the ones that timed out during the network split because the client was unable to decide whether the leader had replicated the request. By assigning these to the indeterminate state, we ensure that there are no false positives regarding consistency violations. Figure 4 points out a natural availability degradation—from 74.53% in the baseline to 69.89% during partitions—as the minority nodes choose not to process writes, favoring consistency over availability (CP in the CAP theorem).

6.4 Bonus Recovery

The recovery script confirmed that committed key-value pairs survive a hard crash and restart. The script outputs `Recovery verification passed: all committed keys survived crash-restart.` is successful. This empirically supports the durable state recovery capabilities of the RocksDB integration.

7 Discussion

7.1 Design Decisions

We were able to bring this project in across the finish line through several key design decisions. First, we implemented a lightweight HTTP shim that greatly improved observability and testability. Next, we implemented read-via-log, which is a clean linearizable read guarantee that is sufficient for preventing stale reads without requiring complex logic. Finally, we kept the Maelstrom adapter in its own component, thus decoupling the benchmarking protocol from the server's internal machinery.

7.2 Challenges and Solutions

There were several implementation challenges that we faced in this project. Indeterminate operations were difficult to implement at first, but we nailed this one by implementing explicit

timeout and disconnect mappings in the shim. Understanding the CAS failure semantics of the Maelstrom implementation meant that we had to treat precondition failures as semantic failures instead of system failures. As discussed in Section 4.3, managing the case of a crash of the leader in the context of the leader-pinned shim meant that we had to implement robust restart and connection drop logic for the shim itself. Finally, we were able to manage the timing instability in our bash scripts for recovery by implementing bounded retries and waiting for substantial periods of time to allow Docker containers to start up properly.

7.3 Threats to Validity

Regarding internal validity: should the adapter misclassify operation outcomes, this could lead to distortion in the checker's input. This was addressed with rigorous mapping logic. Also, due to the nondeterministic nature of our shell scripts, there could be variability in fault timing, thus differing transient availability.

Regarding external validity: while we strictly test with 3 nodes, it is possible that results might not be applicable to larger and more dispersed systems. Our fault models include partition and crash restart faults, but severe clock skew and Byzantine faults are not considered in this project.

7.4 Limitations and Future Work

Although we verified the safety of the OmniPaxos KV implementation with respect to crash recovery and partition faults, there are many avenues for future work.

First, to improve the performance of reads, we might add support for *Read-Index* or *Leader Leases* in the future. Currently, *Read-via-Log* ensures strict linearizability but incurs the cost of a full consensus round trip. Leader leases would allow the leader to serve linearizable reads directly from its local state machine by confirming its leadership with a heartbeat quorum.

Second, we should extend the range of faults and scale up the cluster size. So far, Nemesis has focused on large-scale network partitions and process crashes. Future test campaigns should also include micro-faults such as clock skew and asymmetric network delays. Stress testing larger cluster sizes (e.g., 7 or 11 nodes) would provide deeper insights into how the protocol behaves under heavy contention and message amplification.

Finally, future versions of this pipeline should include latency percentiles (e.g., P95 and P99) to empirically quantify how performance degrades with increasing overheads while maintaining strong consistency in the presence of adversarial faults.

7.5 Lessons Learned

The important point to take away from all of this is that, in addition to the basic replication logic, the interface behavior is also critical to the verification of consistency.

8 Conclusion

This report presents a complete implementation and evaluation of the OmniPaxos KV system with respect to Jepsen-style adversarial testing. We implemented a complete test harness stack, including an HTTP shim, a protocol adapter, as well as automated "nemesis" scripts. We were able to show linearizability for both baseline as well as harsh fault test campaigns, as well as durable recovery with our use of persistent storage. To the limits of our test environment, settings, and circumstances, we can confidently state that the results provided a strong endorsement of the project hypothesis: OmniPaxos KV does, in fact, maintain linearizable behavior even in the face of aggressive network partitioning as well as crash/restart scenarios.

Acknowledgments

A heartfelt "thanks" to the entire teaching staff of the ID2203 VT26 Distributed Systems course. Special thanks to our Course Examiner and Teacher, Paris Carbone, for his enlightening lectures, as well as his guidance in completing our advanced course project. We also gratefully acknowledge Harald Ng and Sonia-Florina Horchidan, our Teaching Assistants, for their support in the labs as well as for providing the foundational repository for our project to be built upon.

References

[1] Kyle Kingsbury. Jepsen: Distributed systems safety research. https://jepsen.io, 2024.

[2] Kyle Kingsbury. Maelstrom: A workbench for learning distributed systems. https://github.com/jepsen-io/maelstrom, 2024.

[3] Kyle Kingsbury. Computational techniques in knossos. *Aphyr Blog*, 2014.

A Appendix A: Repository Structure

```
omnipaxos-kv/
|- src/
| |- server/ # OmniPaxos KV server, network, database
| |- shim/ # HTTP testable API
| |- maelstrom_node/ # protocol bridge for Maelstrom
| |- common.rs # shared commands/messages
|- build_scripts/
| |- run-maelstrom-lin-kv.sh
| |- nemesis.sh
| |- run-nemesis-test.sh
| |- verify-recovery.sh
| |- docker-compose-nemesis.yml
|- VERIFICATION_RESULTS.md
|- BONUS_RECOVERY_RESULTS.md
```

B Appendix B: Pseudocode Algorithms

Algorithm 1: Shim Send Command

Input: Command to be sent to the cluster

Output: Response message or Indeterminate Error

```
id ← next_id++;
pending[id] ← oneshot_sender;
if send Append(id, command) fails then
    remove pending[id];
    return DISCONNECTED (indeterminate);
result ← wait_response_with_timeout(id, T);
if response received then
    return response;
else
    remove pending[id];
    return TIMEOUT (indeterminate);
```

Algorithm 2: Adapter Dispatch

```
Input: Maelstrom JSON request
Output: Response to Maelstrom
if timeout or disconnect from shim then
    return error code 14 (indeterminate);
if type == read then
    return shim_get();
if type == write then
    return shim_put();
if type == cas then
    r ← shim_cas();
    if r.applied then
        return cas_ok;
    else
        return error classified by current value (code 20 or 22);
```

Algorithm 3: Nemesis Loop

```
Input: Fault mode (partition, crash, mixed)
while true do
    action ← mode;
    if mode == mixed then
        action ← random(partition, crash);
    if action == partition then
        partition_start(partition_mode);
        sleep(fault_duration);
        partition_stop();
    else
        crash(target);
        sleep(fault_duration);
        resume();
    sleep(interval);
```

Algorithm 4: Verify Recovery

```
Input: Target node to crash
start cluster;
write baseline keys;
crash target node;
wait for stabilization;
resume target node;
wait for recovery;
read all keys with bounded retries;
assert all read values match expected baseline values;
```

C Appendix C: Reproducibility Commands

```
# Baseline
docker compose -f build_scripts/docker-compose-nemesis.yml up -d --build
MAELSTROM_DIR="$HOME/maelstrom" SHIM_URL="http://127.0.0.1:3000" \
  ./build_scripts/run-maelstrom-lin-kv.sh
docker compose -f build_scripts/docker-compose-nemesis.yml down

# Partition-only
NEMESIS_MODE=partition PARTITION_MODE=majority-minority \
  ./build_scripts/run-nemesis-test.sh

# Crash-only
NEMESIS_MODE=crash CRASH_TARGET=s3 ./build_scripts/run-nemesis-test.sh

# Mixed
NEMESIS_MODE=mixed PARTITION_MODE=majority-minority CRASH_TARGET=s3 \
  ./build_scripts/run-nemesis-test.sh

# Bonus recovery
CRASH_TARGET=s1 KEY_COUNT=8 ./build_scripts/verify-recovery.sh
```